# CLIMATE TIME-SERIES SORTING AND RANKING

Advanced Data Structures (ADS) – C++ Project

Subject: Advanced Data Structures

Programming Language: C++

Algorithms: Quick Sort and Merge Sort

Technique: Divide and Conquer

## 1. ABSTRACT

Climate data plays an important role in understanding environmental changes and weather conditions. Managing large amounts of climate information requires efficient sorting and analysis techniques.

The Climate Time-Series Sorting and Ranking project is developed using C++. It uses two important sorting algorithms, Quick Sort and Merge Sort, to organize climate records based on temperature and rainfall.

The system identifies the highest and lowest temperatures and compares the execution time of both algorithms. It demonstrates the practical application of sorting techniques in environmental data analysis.

## 2. INTRODUCTION

Climate change and environmental monitoring have become important global concerns. Climate records such as temperature and rainfall are collected from different locations over time.

Organizing these records manually is difficult and time-consuming.

This project provides a simple C++ application to sort and analyze climate records using efficient sorting algorithms.

The project demonstrates the use of Divide and Conquer techniques through Quick Sort and Merge Sort.

# 3. PROBLEM STATEMENT

Climate records contain important information such as temperature and rainfall.

Sorting and ranking these records manually becomes difficult when the amount of data increases.

The main problem is to develop an efficient system that can organize climate records, identify extreme temperatures and compare sorting algorithm performance.

# 4. OBJECTIVES

- Implement Quick Sort using C++.

- Implement Merge Sort using C++.

- Sort climate records by temperature.

- Sort climate records by rainfall.

- Identify maximum and minimum temperatures.

- Compare execution times of sorting algorithms.

- Understand Divide and Conquer techniques.

- Analyze algorithm time complexity.

# 5. SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

- Operating System: Windows / Linux

- Programming Language: C++

- Compiler: GCC / G++

- IDE: Visual Studio Code

Hardware Requirements

- Processor: Intel or equivalent

- RAM: Minimum 4 GB

- Storage: Minimum 500 MB free space

# 6. DATA STRUCTURE USED

Structure

The project uses a C++ structure named "ClimateRecord".

It stores three important fields:

1. City – Name of the city.

2. Temperature – Temperature recorded in Celsius.

3. Rainfall – Rainfall recorded in millimeters.

A vector is used to store multiple climate records dynamically.

# 7. ALGORITHMS USED

## QUICK SORT

Quick Sort is a Divide and Conquer sorting algorithm.

It selects an element called the pivot and partitions the remaining elements into two groups.

One group contains smaller elements, while the other contains larger elements.

- ✓ Steps:

1. Select a pivot element.

2. Partition the array around the pivot.

3. Place smaller elements before the pivot.

4. Place larger elements after the pivot.

5. Recursively sort both partitions.

Time Complexity:

- Best Case: O(n log n)

- Average Case: O(n log n)

- Worst Case: $O(n^2)$

## MERGE SORT

Merge Sort is another Divide and Conquer algorithm.

It divides the array into smaller parts, sorts each part and combines them into a single sorted array.

- ✓ Steps:

1. Divide the array into two halves.

2. Recursively divide each half.

3. Sort the smaller sections.

4. Merge the sorted sections.

5. Repeat until the complete array is sorted.

Time Complexity:

- Best Case: O(n log n)

- Average Case: O(n log n)

- Worst Case: O(n log n)

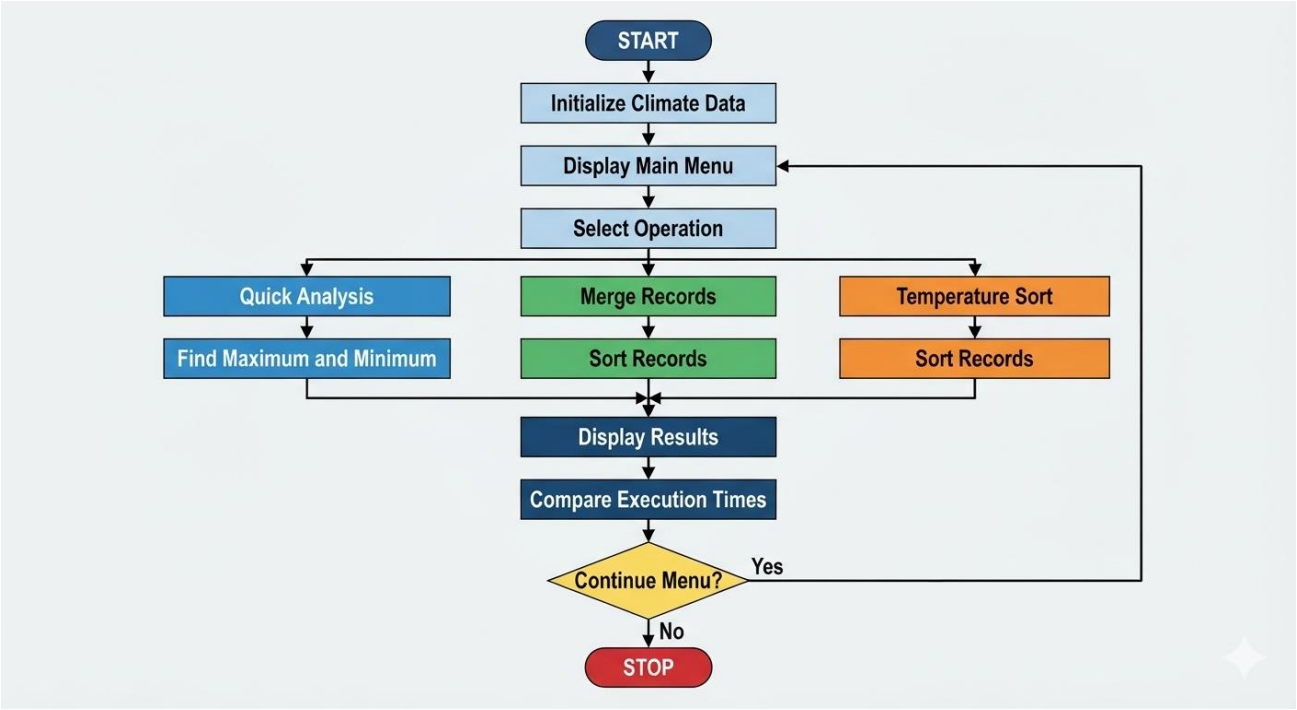
# 8. SYSTEM METHODOLOGY

The project follows these steps:

- ❖ Start the application.
- ❖ Initialize climate records.
- ❖ Display the main menu.
- ❖ Select the required operation.

- ❖ Choose temperature or rainfall sorting.
- ❖ Execute Quick Sort or Merge Sort.
- ❖ Display sorted climate records.
- ❖ Identify highest and lowest temperatures.
- ❖ Compare algorithm execution times.
- ❖ Display the final results.

## 9. FLOWCHART




## 10. IMPLEMENTATION

The project is implemented using C++ programming language.

- Climate Data Initialization

Initializes climate records containing city names, temperature and rainfall values.

- Display Records

Displays all available climate records in a tabular format.

- Quick Sort Module

Sorts climate records using the Quick Sort algorithm.

- Merge Sort Module

Sorts climate records using the Merge Sort algorithm.

- Temperature Analysis

finds the highest and lowest temperature records using maximum and minimum element operations.

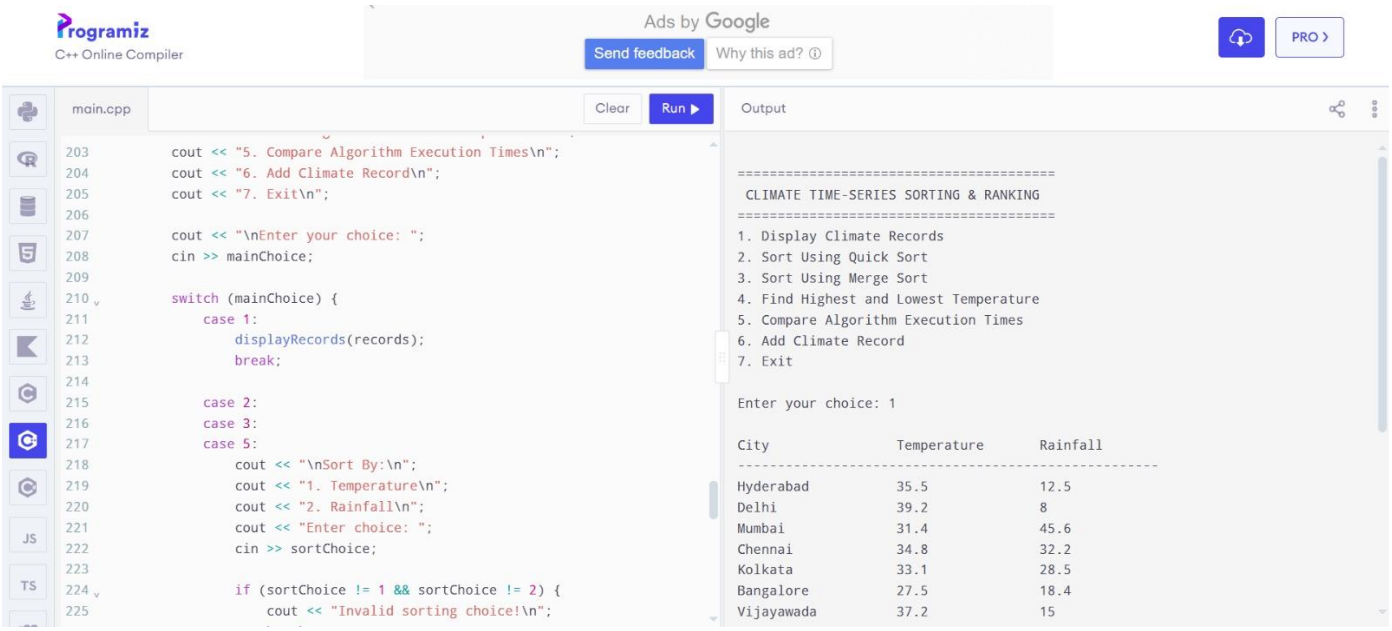
- Performance Comparison

Measures the execution time of Quick Sort and Merge Sort using the C++ Chrono library.

- Add Climate Record

Allows users to enter new climate records.

# 11. OUTPUT AND RESULTS




Algorithm| Best Case| Average Case| Worst Case

Quick Sort| O(n log n)| O(n log n)| $O(n^2)$

Merge Sort| O(n log n)| O(n log n)| O(n log n)

## 13. ADVANTAGES

- Simple and user-friendly interface.

- Efficient sorting of climate records.

- Easy identification of extreme temperatures.

-Practical implementation of ADS concepts.

- Performance comparison between algorithms.

- Useful for understanding climate information.

## 14. REAL-WORLD APPLICATIONS

- Climate data analysis.

- Rainfall monitoring.

- Environmental monitoring.

- Historical weather record management.

- Identification of extreme weather events.

- Climate research and analysis.

## 15. FUTURE ENHANCEMENTS

- Import climate records from CSV files.

- Add graphical climate visualization.

- Develop a web-based interface.

- Support larger climate datasets.

- Generate detailed climate analysis reports.

- Implement additional sorting algorithms.

# 16. CONCLUSION

The Climate Time-Series Sorting and Ranking project demonstrates the practical application of Quick Sort and Merge Sort algorithms using C++.

The system successfully organizes climate records based on temperature and rainfall, identifies extreme temperatures and compares algorithm execution times.

This project improves understanding of Divide and Conquer techniques and demonstrates how Advanced Data Structures can be applied to real-world environmental data processing.